Spring Framework: IoC, веб, данные и AOP

Информационные системы · Конспект лекции

Роль Spring

Spring — открытый Java-фреймворк с центральной идеей IoC/Dependency Injection. В отличие от платформы спецификаций, Spring предоставляет собственную экосистему модулей. Обычно Spring Core связывает компоненты, Spring MVC обслуживает HTTP, Spring Data упрощает доступ к данным, Spring Security отвечает за безопасность, а Spring Boot собирает совместимую конфигурацию и автоматическую настройку.

ApplicationContext и бины

ApplicationContext — контейнер Spring: создаёт бины, внедряет зависимости, публикует события и обрабатывает ресурсы. Бины регистрируют через @Configuration/@Bean или обнаруживают через @ComponentScan и стереотипы @Component, @Service, @Repository, @Controller. Зависимости лучше задавать через конструктор: они явны, обязательны и облегчают неизменяемость. При нескольких реализациях помогают @Qualifier или @Primary.

Жизненный цикл и конфигурация

Контейнер создаёт бин, внедряет зависимости, выполняет post-processing и callback-и и затем управляет завершением. @PostConstruct подходит для инициализации после внедрения; @PreDestroy — для освобождения ресурсов. Конфигурационные значения задают внешне (properties/yaml) и связывают с типизированными настройками, например через @ConfigurationProperties. Это позволяет разделять код и настройки среды.

Spring Web MVC

MVC делит ответственность между моделью, контроллером и представлением. DispatcherServlet принимает HTTP-запрос, находит обработчик, передаёт параметры и формирует ответ. Контроллеры задают маршруты и методы; Model переносит данные во ViewResolver и шаблон. Для REST обычно используют контроллеры, возвращающие данные, и явно описывают HTTP-метод, статус, формат и обработку ошибок. Валидация входа — часть границы API.

Spring Boot

Boot ускоряет запуск приложения через starters, согласованные версии зависимостей, автоконфигурацию и встроенный сервер. Автоконфигурация условна: применяются настройки, подходящие зависимостям и собственным бинам приложения. Её следует понимать и проверять, а не воспринимать как скрытую магию. Внешние параметры позволяют одним артефактом запускать приложение в разных средах.

Spring Data и бизнес-слои

Репозиторий предоставляет операции доступа к данным и может генерировать реализацию по интерфейсу/имени метода. Сервисный слой координирует бизнес-сценарии и задаёт транзакционную границу; контроллер отвечает за HTTP, а репозиторий — за хранение. Не стоит помещать бизнес-правила в контроллер или выдавать наружу управляемые сущности без учёта связей и загрузки.

AOP и прокси

AOP выделяет сквозные задачи: транзакции, журналирование, безопасность, метрики. Pointcut выбирает места выполнения, advice задаёт действие до/после/вокруг вызова. В Spring AOP часто применяется через прокси, поэтому перехватываются вызовы через прокси; внутренний self-invocation может обойти аспект. Это объясняет некоторые неожиданные эффекты аннотаций @Transactional.

Что запомнить

Spring связывает приложение через контейнер бинов. Boot упрощает конфигурацию, MVC организует обработку HTTP, Data стандартизирует доступ к данным, AOP применяет сквозные правила. Делите ответственность по слоям и помните, что декларативное поведение часто реализуется прокси-контейнером.

Термины

IoC · ApplicationContext · Spring MVC · Spring Boot · Spring Data · AOP

Составлено по материалам исходной лекции. Проверьте поколение API и особенности используемой СУБД/контейнера.